

PaycheckCalc Roadmap

Current Repository Rewrite

Static review date: June 17, 2026

Bottom-line diagnosis

The old roadmap is now materially stale. PaycheckCalc is no longer just a paycheck calculator with future budgeting ideas. The repository now contains a paycheck-first finance platform: W-2 paycheck calculation, gross-up, bonus/supplemental wages, self-employment estimates, annual projection, saved paycheck comparison, budget tracking, recurring bills, savings goals, reports, optional accounts, and sync. The next roadmap should shift from "add big features" to "harden accuracy, version tax data, protect financial data, polish scenarios, and prepare monetization."

Review scope and limits

This rewrite is based on a static repository review through the GitHub connector plus the uploaded prior roadmap PDF. I reviewed project structure, README/wiki documentation, Core calculator architecture, MAUI/Blazor UI surfaces, Shared sync/contracts, API startup/endpoints, migrations, budget/reporting files, representative tests, and important implementation snippets. I did not run the solution locally, did not run tests, and did not inspect every JSON tax table byte-for-byte or binary asset. Treat this as a product/architecture roadmap, not a formal code audit or tax certification.

1. Executive Summary

The roadmap should now be organized around three realities: accuracy is the moat, privacy is the release gate, and paycheck data is the engine for every future finance feature.

- The Core engine is still the strategic center: PayCalculator, FICA, IRS 15-T, state withholding, deductions, explanations, annual projection, gross-up, bonus, self-employment, budgeting, and reports all live outside UI-specific code.
- The UI scope is broader than before: MAUI now has Inputs, Results, Paychecks, Budget, and Account tabs. Blazor has calculator, budget, exports, saved paycheck/account panels, and state landing pages.
- The old roadmap items for basic Budget V1, recurring bills, savings goals, budget reports, budget sync, API migrations, database documentation cleanup, bonus calculator, and self-employment estimator should be marked complete or moved from feature discovery to hardening/polish.
- The biggest unresolved product risk is not lack of features. It is trust: tax-year versioning, source metadata, source-backed exports, privacy controls, account deletion, data export, sync failure UX, and release-mode security still matter more than adding another calculator.
- The strongest next product move is not a Mint clone. It is paycheck-based planning: profiles, job comparison, W-4 optimizer, paycheck calendar, bills assigned to paydays, safe-to-spend, and 30/60/90-day cashflow forecasting.

Area	Current repository state	Roadmap consequence
Paycheck accuracy	Strong architecture with IRS 15-T, FICA, all states + DC, deductions, explanations, gross-up, bonus, and self-employment.	Protect and prove accuracy before adding more breadth.
Budgeting	Monthly budget tracker, budget methods, categories, transactions, recurring bills, savings goals, reports, local storage, sync model.	Do not roadmap this as future V1. Roadmap paycheck-based planner as the next budget evolution.
Sync/API	Optional accounts, Identity endpoints, paycheck sync, budget sync, PostgreSQL, migrations for PostgreSQL startup.	Move from prototype sync to production privacy/security/recovery controls.
Monetization	Entitlement abstraction exists; default provider effectively acts as placeholder/free access.	Build actual entitlement/billing strategy after release blockers are removed.
Release readiness	.NET 11 preview stack, MAUI platform considerations, no formal store/privacy package yet.	Treat packaging, policy, and privacy as P0, not later polish.

2. What the Repository Currently Is

2.1 Solution architecture

Project	Role in current repository	Roadmap meaning
PaycheckCalc.Core	UI-agnostic domain layer containing paycheck, tax, pay, gross-up, annual projection, bonus, self-employment, budgeting, and report engines.	Must remain clean. Do not let UI, HTTP, store billing, or platform code leak into Core.
PaycheckCalc.App	.NET MAUI Android/Windows app using MVVM, Shell tabs, local stores, exports, print, sync, account, budget UI.	Primary mobile/desktop product surface. Needs parity, polish, privacy controls, and release packaging.
PaycheckCalc.Blazor	Blazor Server web app with calculator, budget, saved paychecks, account/sync, exports, print, and state landing pages.	Good demo/web surface and SEO/testing surface. Should be kept aligned with MAUI but not allowed to hide MAUI gaps.
PaycheckCalc.Shared	DTOs, JSON options, API client, mergers, tombstones, store abstractions, entitlements, sync services.	Correct place for cross-client persistence contracts and conflict behavior.
PaycheckCalc.Api	ASP.NET Core Identity API with paycheck and budget sync endpoints, EF Core, PostgreSQL, migrations.	Optional sync backend. Needs production hardening before public user accounts.
PaycheckCalc.Tests	xUnit suite across calculators, budget,	The test suite is now a product asset.

	sync, exports, state logic, snapshot JSON, and API integration.	Expand it into an accuracy proof matrix.
--	---	--

Architecture verdict

The separation is good and should be defended. Core is the payroll/finance engine; App and Blazor are presentation; Shared is portable contracts/sync; API is optional persistence. This is the right classic architecture. Old-school separation of concerns still wins because money math mixed with UI code is how calculators become haunted houses.

2.2 Core paycheck engine

- Standard paycheck mode supports hourly pay with regular/overtime hours and salary pay by annual or per-period basis.
- PayCalculator calculates gross, pre-tax deductions, post-tax deductions, FICA-taxable wages, federal taxable wages, federal withholding, state taxable wages, state withholding, disability/paid-leave premiums, and net pay.
- Deductions are now strong enough to support serious paycheck modeling: dollar or percentage amounts, pre-tax or post-tax, and independent flags for reducing federal taxable wages, state taxable wages, and FICA wages.
- FICA logic handles Social Security, Medicare, Additional Medicare, wage base, and YTD wage inputs at the Core model level.
- Federal withholding implements IRS Publication 15-T 2026 percentage method using W-4 filing status, Step 2, Step 3, Step 4(a), Step 4(b), and Step 4(c).
- State withholding is plugin/registry based and uses dynamic state schemas for state-specific fields.
- All major output lines are connected to Show Your Work explanation models.

2.3 Current calculation modes

Mode	Current implementation state	Roadmap treatment
Standard paycheck	Core and both front-ends support regular W-2 paycheck calculation.	Harden and polish. This remains the flagship experience.
Gross-up	Core solver exists and UI supports target-net calculations.	Keep as high-visibility differentiator; later add gross-up bonus/scenario templates.
Bonus / supplemental wage	BonusCalculator exists, composes federal supplemental withholding, FICA, and state supplemental withholding.	Move from future idea to polish/test/UX phase.
Self-employment / 1099	SelfEmploymentCalculator estimates SE tax, state income tax, quarterly payments, and explanations. Federal income tax intentionally out of scope.	Keep disclaimers obvious; future roadmap can add federal income-tax estimator or W-2 vs 1099 comparison.

2.4 State tax engine

- All 50 states plus District of Columbia are represented through IStateWithholdingCalculator registrations.
- State-specific input schemas live in PaycheckCalc.Core/Data/Schemas and are consumed by both front-ends.
- No-income-tax states are handled either by a shared adapter or dedicated calculators for states with employee-paid assessments, such as Washington WA Cares.
- Dedicated income-tax calculators now cover the income-tax jurisdictions instead of relying on a generic one-size-fits-most path.
- JSON-backed tax data exists for federal 15-T, Arkansas, California, Colorado, Connecticut, Oklahoma, and state supplemental wages.
- State disability and paid-leave premiums are surfaced separately for California, Colorado, Connecticut, and Washington.

Important state-engine note

StateCalculatorRegistry.Register replaces prior registrations for the same state. The current StateTaxConfigs2026 file appears to be comment/reference infrastructure rather than active production config entries, so it likely does not override dedicated calculators today. Still, add guard tests. This is exactly the sort of quiet issue that can become a tax accuracy bug later.

2.5 Budgeting and reports

Budget feature	Current state	Next treatment
Monthly budget tracker	Implemented in Core and surfaced in MAUI/Blazor.	No longer future V1; polish and integrate with paycheck profiles.
Budget methods	Custom, 50/30/20, Zero-Based, and Envelope methods exist.	Add better onboarding and templates.
Categories and transactions	BudgetCategory and BudgetTransaction	Improve UX, edit flows, filtering, and date

	support monthly planning and expense tracking.	handling.
Recurring bills	RecurringBill and recurrence monthly-equivalent normalization exist.	Evolve into bill-to-paycheck assignment.
Savings goals	SavingsGoal with target date and monthly contribution exists.	Integrate into safe-to-spend and sinking funds.
Budget reports	BudgetReportCalculator plus CSV/PDF export paths exist; report UI is entitlement-gated.	Keep gated later, but make the free/report boundary explicit.
Budget sync	Budget, transaction, recurring bill, and savings goal sync/tombstones exist.	Harden privacy, merge UX, export/delete flows.

2.6 Sync, accounts, and persistence

- Accounts are optional. MAUI persists saved paychecks and budgets locally; Blazor uses circuit/session storage for anonymous use; authenticated users can sync through the API.
- The API maps Identity account endpoints plus authorized paycheck and budget sync endpoints.
- Saved paycheck sync uses snapshots and tombstones with deterministic last-write-wins merge behavior.
- Budget sync is split into four independent collections: budgets, transactions, recurring bills, and savings goals.
- The API now uses PostgreSQL via Npgsql and applies EF Core migrations for PostgreSQL startup. EnsureCreated remains for non-PostgreSQL test paths, which is acceptable.
- Known limitations still include lack of server-side rate limiting, plain local HTTP development defaults, and client-timestamp clock-skew conflict risk.

3. What Has Already Been Added Since the Prior Roadmap

Prior roadmap item / risk	Current repository state	New status
Budget V1 - monthly budget tracker	Monthly budget tracker exists with income normalization, categories, transactions, and budget methods.	Done; polish and evolve.
Recurring bills	RecurringBill model, monthly-equivalent normalization, UI, persistence, and sync are present.	Done; next step is bill-to-paycheck assignment.
Savings goals	SavingsGoal model, target-date contribution calculation, UI, persistence, and sync are present.	Done; integrate into safe-to-spend.
Budget reports	BudgetReportCalculator, spend-by-category, budget-vs-actual, CSV/PDF export paths are present.	Mostly done; entitlement/billing not wired.
Budget sync	Budget sync endpoint and shared merger cover budgets, transactions, recurring bills, savings goals, and tombstones.	Done; harden sync UX/privacy.
API SQLite/PostgreSQL doc mismatch	README/wiki now describe PostgreSQL; Program.cs uses Npgsql.	Resolved.
Replace production EnsureCreated	Production PostgreSQL path now uses Database.Migrate(); EnsureCreated remains only for non-Npgsql test path.	Resolved for production intent.
MAUI annual projection	MAUI Results page contains Per Paycheck/Annual tabs and annual projection display/export wiring.	Done.
Bonus/gross-up planner	Gross-up already existed; bonus/supplemental wage mode now exists.	Gross-up done; bonus mode added.
Self-employment / 1099 calculator	SelfEmploymentCalculator now estimates SE tax, state income tax, quarterly payments, and explanations.	Added; caveats must stay clear.
Entitlement abstraction	IEntitlementProvider and FreeEntitlementProvider exist.	Skeleton done; paid implementation remains.
Class diagrams/wiki	Docs/wiki and class diagram exist for architecture, tax engine, state coverage, budgeting, and sync.	Improved; continue keeping docs tied to code.

Roadmap reset

Do not keep completed features in the next roadmap as if they still need to be invented. That creates a fake backlog and hides the real work. The new backlog is about versioning, proof, safety, release readiness, scenario UX, and monetization.

4. Remaining Technical and Product Risks

Priority	Risk	Why it matters	Recommended action
P0	Tax year still not first-class in PaycheckInput	The app is 2026-centric. Saved results and future tax years need versioned rules. Old saved paychecks must not silently recalculate under newer tax data.	Add TaxYear to PaycheckInput, saved snapshots, exports, CommonWithholdingContext flow, and tax-data resolver abstractions.
P0	Source metadata is not centralized	Accuracy is the product moat, but users need visible proof. Source references inside explanations are useful but not a full metadata system.	Add TaxSourceMetadata / TaxDataVersion per calculator and expose an Accuracy & Sources screen.
P0	Blazor state picker still uses all enum values	Docs say registry-supported states drive pickers, but code shows Enum.GetValues<UsState>(). It is safe only while every enum is supported.	Use StateRegistry.SupportedStates consistently. Add a regression test or component-level guard.
P0	Blazor decimal validation/parsing mismatch remains	Validate() uses culture-sensitive decimal.TryParse while GetValue() uses invariant culture. Some inputs can validate one way and calculate another.	Normalize parsing with invariant culture or typed InputNumber<decimal>. Add tests for 1.25, 1,25, blank, invalid, and negative.
P0	MAUI YTD FICA inputs appear absent	Core supports YTD Social Security/Medicare wages and Blazor exposes them. MAUI parity matters for high-income edge cases.	Add MAUI fields with help text and map to PaycheckInput.
P0	Financial privacy controls are incomplete for public release	Saved paychecks and budgets are sensitive financial data.	Add delete account, export data, delete local data, privacy policy, terms, and sync disclosure before public sync.
P0	API production hardening still needed	Accounts and sync create attack surface and support burden.	Add rate limiting, HTTPS-only production guidance, safe logging, friendly errors, retry policy, and monitoring.
P1	.NET 11 preview launch risk	Preview SDK/framework can complicate Android/Windows store readiness.	Decide release target and verify publish/sign/store pipelines early.
P1	Entitlement/billing is placeholder	FreeEntitlementProvider is useful for development but not monetization.	Implement entitlement source, feature flags, local/pro/store billing flow.
P1	Self-employment federal income tax excluded	Users may mistake take-home before federal income tax for full tax estimate.	Keep explicit warnings in UI/export; later add optional federal income-tax estimator.

5. Strategic Product Direction

Recommended positioning:

Product promise

An accurate, transparent paycheck-first finance app that shows how your take-home pay is calculated, then uses that paycheck model to plan bills, budgets, job changes, withholding, bonuses, and self-employment income.

Avoid positioning	Use positioning instead	Reason
Generic personal finance app	Paycheck-first finance planner	Generic finance apps are crowded; paycheck accuracy is the differentiator.
Budget app with paycheck calculator	Payroll-accuracy engine with budget/planning tools	Core math and state coverage are the hard part. Lead with that.
100% accurate paycheck calculator	Source-backed 2026 paycheck estimates with transparent calculations	Defensible wording avoids legal/support traps.
Tax app	Withholding and take-home pay planner	Full tax filing is not the product and would explode scope.

5.1 Ideal users

- Hourly workers with overtime, variable hours, bonuses, deductions, or paycheck-to-paycheck budgeting needs.
- Salaried employees comparing jobs, raises, bonuses, deductions, W-4 changes, or benefit elections.
- Workers in states with complicated withholding or paid-leave/disability premiums.
- People adjusting W-4 withholding to avoid surprises at tax time.
- Independent contractors comparing W-2 vs 1099 take-home and quarterly-payment exposure.
- Small business owners, HR users, and payroll admins doing sanity checks.

6. Updated Roadmap

Milestone 1 - Trustworthy Calculator Hardening

Priority	Task	Details / acceptance criteria
P0	Add TaxYear as a first-class input	PaycheckInput.TaxYear, CommonWithholdingContext, snapshots, exports, PDF/CSV, sync DTOs, and UI display. Default 2026.
P0	Add TaxDataVersion and source metadata	Federal/state/supplemental/self-employment calculators expose source name, method, table/publication, tax year, verified date, and calculator version.
P0	Build Accuracy & Sources screen	Show tax year, federal method, state method, last verified, data version, rounding rules, and caveats. Add export copy.
P0	Fix Blazor state picker	Replace Enum.GetValues<UsState>() with StateRegistry.SupportedStates. Add test/guard.
P0	Fix Blazor decimal parsing	Use invariant parsing consistently or typed InputNumber<decimal>. Add culture/invalid-value tests.
P0	Add MAUI YTD FICA fields	Expose YTD Social Security and Medicare wages in Inputs page and map into PaycheckInput.
P0	Improve input validation	Reject/flag invalid negative values, impossible paycheck number, percentages > expected ranges, and blank required state fields.
P1	Expand official golden tests	Federal 15-T bracket boundaries, state examples where official examples exist, FICA cap crossing, Additional Medicare threshold crossing, supplemental wage tests.
P1	Export accuracy evidence	PDF/CSV should include tax year, method, data source, and key caveats.

Milestone 2 - Release-Ready Sync, Privacy, and Security

Priority	Task	Details / acceptance criteria
P0	Privacy policy and terms	Required before public account/sync. Explain what data is stored, where, and why.
P0	Delete account	Endpoint and UI flow that deletes account and server-side sync data.
P0	Export user data	User can export saved paychecks and budget data in a portable format.
P0	Delete local data	MAUI can clear local paychecks, budgets, bills, transactions, goals, tokens, and corrupt backup files if present.
P0	Sync disclosure and control	Make sync optional and clearly explain what is stored. Avoid account creation as a requirement.
P0	Rate limiting	Add rate limits to account and sync endpoints.
P0	Production HTTPS configuration	Document and enforce HTTPS outside local development.
P1	Safe logging policy	No raw wages, net pay, budget amounts, or token values in normal logs.
P1	Sync conflict/clock skew UX	Surface last-sync status, failures, retry, and conflict caveats.
P1	App lock / biometric unlock	Optional local privacy control for saved financial data.

Milestone 3 - Profiles, Scenarios, and Saved Paycheck UX

Priority	Task	Details / acceptance criteria
P1	Paycheck profiles	Save reusable job/pay setup: state, frequency, W-4, state form, pay type, typical hours, deductions, YTD wages.
P1	Edit/duplicate saved paychecks	Turn saved results into reusable scenarios, not dead snapshots.
P1	Scenario templates	Hourly job, salary job, overtime week, bonus, gross-up bonus, second job, new job offer, 401(k) change, HSA/FSA change, no-income-tax state, California SDI, Washington WA Cares.
P1	What changed? comparison	When a field changes, show change in net pay, total tax, take-home percent, annualized net, and effective tax rate.
P1	Job offer comparison mode	Compare two jobs with different states, pay frequencies, deductions, overtime assumptions, and annual net.
P2	Share/copy result	Mobile share sheet and copy-to-clipboard result summary.
P2	Saved paycheck search/filter	Filter by state, pay type, profile, updated date, and net/gross range.

Milestone 4 - Paycheck-Based Budgeting and Payday Planning

Priority	Task	Details / acceptance criteria
P1	Paycheck calendar	First paycheck date, frequency, future paydays, 27-biweekly and 53-weekly calendar support.
P1	Bill-to-paycheck assignment	Assign recurring bills to specific paychecks instead of only monthly equivalents.
P1	Safe-to-spend result	Available money until next payday after bills, savings goals, and envelopes.
P1	30/60/90-day cashflow forecast	Projected income, bills, savings, spend, and shortfall warnings.

P1	Three-paycheck month detection	Flag months with extra paychecks and let users assign the extra money.
P2	Sinking funds	Car repair, holidays, annual insurance, property tax, annual subscriptions, medical.
P2	Budget rollover	Envelope/category balances roll into the next month or next paycheck period.
P3	CSV/OFX import	Only after manual budgeting is solid. Do not start with bank integration.

Milestone 5 - Optimization Tools

Priority	Tool	Why it belongs here
P1	W-4 optimizer	Uses annual projection and withholding inputs to suggest Step 3 / Step 4(c) changes toward a target refund/owed amount.
P1	Refund/owed estimator	Natural extension of current annual projection, but must be labeled as withholding estimate, not full tax filing.
P1	401(k) / HSA / FSA impact calculator	Uses existing deduction flags to show net-pay impact and annual tax effect.
P1	Roth vs traditional paycheck impact	Direct fit for deduction engine.
P1	Raise calculator	High-search, easy to explain, uses existing payroll engine.
P1	Overtime planner	Very useful for hourly workers; shows take-home from extra hours.
P2	1099 vs W-2 comparison	Build on self-employment estimator plus W-2 paycheck engine. Must include benefit/tax caveats.
P2	Bonus gross-up planner	Combine bonus/supplemental mode with gross-up target.

Milestone 6 - Monetizable Pro

Priority	Task	Details / acceptance criteria
P1	Real entitlement provider	Replace default free placeholder with local/dev/test and production entitlement implementations.
P1	Feature flags	Central list of gated features and free limits.
P1	Pricing/paywall screen	Explain Pro value without blocking basic paycheck calculation.
P2	Billing integration	Google Play Billing first if Android is primary; later Microsoft Store/App Store if applicable.
P2	Privacy-safe analytics	Track feature usage without raw wages, net pay, tax amounts, or budget details. Opt out where required.
P2	Crash reporting	Capture platform, version, screen, and exception class without financial payloads.
P2	Store assets	Screenshots for calculator, Show Your Work, state inputs, gross-up, bonus, self-employment, budget, sync, exports.

7. Monetization Recommendation

Tier	Recommended features	Rationale
Free	Basic paycheck calculation, federal/state/FICA, one or limited profiles, basic saved result, gross-up	Free tier must prove trust and generate organic use.

	preview, basic explanations, limited budget categories, limited exports.	
Pro	Unlimited profiles/scenarios, W-4 optimizer, job comparison, full annual projection tools, paycheck calendar, budget planner, reports, unlimited exports, sync, tax-year archive, advanced deductions, bonus/gross-up tools.	Power users pay for planning, optimization, cross-device convenience, and reports.
Lifetime	One-time unlock with clear scope and no promise of forever tax-data updates unless you mean it.	Good for early supporters; be careful with future data-maintenance costs.
B2B later	Embeddable calculator, payroll sanity-check API, state withholding API, HR/job-offer calculator.	Core/UI separation makes this realistic, but not before consumer trust is established.

Ads recommendation

If ads are used, keep them away from results, Show Your Work, exports, and account/privacy flows. A paycheck result screen is a trust moment. Slapping an intrusive ad there makes the app look cheap, and cheap is not the vibe when money math is involved.

8. GitHub Issue Backlog to Create Next

8.1 P0 foundation issues

1. Add TaxYear to PaycheckInput and CommonWithholdingContext; default to 2026.
2. Persist TaxYear in saved paycheck snapshots and sync DTO schema.
3. Include TaxYear and data version in PDF/CSV exports.
4. Stop hardcoding Year: 2026 inside PayCalculator; resolve tax data through a tax-year provider.
5. Add TaxSourceMetadata model and interface for federal/state/supplemental/self-employment calculators.
6. Create Accuracy & Sources screen in MAUI and Blazor.
7. Fix Blazor state picker to use StateRegistry.SupportedStates.
8. Normalize Blazor decimal parsing/validation to invariant culture or typed InputNumber.
9. Add MAUI YTD Social Security and YTD Medicare wage inputs.
10. Add privacy policy, terms, account deletion, data export, and local data delete flows.
11. Add rate limiting to account and sync endpoints.
12. Add production HTTPS configuration guidance and safe production defaults.

8.2 P1 accuracy and test issues

13. Add federal IRS 15-T golden tests across all filing statuses, Step 2 branches, and pay frequencies.
14. Add federal bracket-boundary tests.
15. Add FICA Social Security wage-base crossing tests.
16. Add Additional Medicare threshold crossing tests with YTD wages.
17. Add deduction matrix tests: traditional 401(k), Roth 401(k), HSA, FSA, Section 125 medical, post-tax garnishment, union dues, percentage deduction.
18. Add state known-example tests where official examples exist.
19. Add source metadata completeness tests for every calculator.
20. Add export tests confirming tax year/source/version appears in PDF/CSV.
21. Add bonus/supplemental tests around \$1,000,000 federal threshold and regular-method states.
22. Add self-employment tests for W-2 YTD wage interaction and quarterly payment remainder handling.
23. Add registry tests proving every UsState has the intended calculator and no accidental generic override.

8.3 P1/P2 product issues

24. Create paycheck profile model and UI.
25. Add profile templates for common paycheck scenarios.
26. Add edit/duplicate saved paycheck flow.
27. Add What Changed comparison after editing inputs.

28. Add job offer comparison mode.
29. Add paycheck calendar with first-payday setup.
30. Add recurring bill assignment to paychecks.
31. Add safe-to-spend calculation.
32. Add W-4 optimizer prototype using AnnualProjectionCalculator.
33. Add 401(k)/HSA/FSA paycheck impact calculator.
34. Add Roth vs traditional paycheck impact tool.
35. Add raise and overtime calculators.

8.4 P2/P3 monetization/platform issues

36. Implement production IEntitlementProvider and Pro feature flags.
37. Create pricing/paywall UI and free-limit copy.
38. Wire Play Billing or chosen store billing provider.
39. Add privacy-safe analytics with no raw financial amounts.
40. Add crash reporting with financial payload scrubbing.
41. Create release build/signing pipeline for Android and Windows.
42. Create store screenshots and marketing copy tied to source-backed accuracy claims.
43. Add support/export workflow for user data and account deletion requests.

9. Feature Priority Ranking

Rank	Feature	Value	Why
1	Tax-year/data-version infrastructure	Foundational	Everything else breaks historically if tax years are not versioned.
2	Accuracy & Sources screen	Very high	Turns hidden correctness into user trust.
3	Privacy/data deletion/export	Very high	Required for accounts/sync and financial data.
4	W-4 optimizer	Very high	Strong premium feature built on existing annual projection.
5	Paycheck calendar	Very high	Bridges paycheck calculator and real-life budgeting.
6	Job offer comparison	Very high	Highly marketable; users compare net, not just gross.
7	Bill-to-paycheck budgeting	Very high	Makes the budget module uniquely paycheck-first.
8	401(k)/HSA/FSA impact tools	High	Leverages existing deduction engine.
9	Bonus gross-up planner	High	Combines two strong existing modes.
10	Self-employment federal income tax extension	Medium-high	Valuable, but must be carefully scoped.
11	Debt payoff	Medium	Good later budget companion, not core moat.
12	Bank import/sync	Later	Useful but compliance-heavy and distracting before manual planning is excellent.
13	Investment tracker	Much later	Dilutes the product unless paycheck planning is already successful.

10. The Do-Not-Mess-This-Up List

44. Do not weaken Core/UI/API separation.
45. Do not use double or float for money.
46. Do not add budgeting or billing logic inside paycheck calculators.
47. Do not hide tax assumptions, source data, or tax-year rules.
48. Do not make account creation mandatory for basic calculator use.

49. Do not ship sync without account deletion, data export, and local data deletion.
50. Do not recalculate old saved paychecks under a new tax year without explicit user action.
51. Do not market "100% accurate". Use source-backed estimates with transparent calculations.
52. Do not add bank integration before manual paycheck-based budgeting is useful.
53. Do not put aggressive ads near paycheck results, Show Your Work, or exports.
54. Do not let preview framework choices block app store release.
55. Do not let documentation drift from implementation. The docs are now part of the trust story.

Appendix A - Repository Evidence Reviewed

Repository area	Files / evidence sampled	What it confirmed
High-level docs	README.md; docs/wiki/Architecture.md; docs/wiki/Tax-Calculation-Engine.md; docs/wiki/State-Tax-Coverage.md; docs/wiki/Budgeting.md; docs/wiki/Accounts-and-Sync.md	Current features, architecture, tax pipeline, budget scope, sync design, state coverage, docs alignment.
Core models/pay	PaycheckInput.cs; PayCalculator.cs; AnnualProjection.cs; BonusCalculator.cs; SelfEmploymentCalculator.cs; GrossUpCalculator.cs; PayPeriods.cs	Standard paycheck, annual projection, bonus, self-employment, gross-up, pay frequency, deduction/FICA handling.
Core state/federal/FICA	PaycheckCoreServiceCollectionExtensions.cs; StateCalculatorRegistry.cs; StateTaxConfigs2026.cs; state calculator search results; supplemental calculator	State registrations, JSON-backed data, schema provider, supplemental rates, registry behavior, 50 states + DC coverage.
MAUI app	AppShell.xaml; CalculatorViewModel.cs; ResultsPage.xaml; BudgetPage.xaml; BudgetViewModel search/docs; export/print service search results	Five-tab app, budget tab, annual/results tabs, calculation modes, exports, print, local stores, sync wiring.
Blazor app	Calculator.razor; Calculator.razor.cs; Budget.razor search/docs; Program.cs search/docs; export services	Calculator modes, YTD fields, state picker behavior, parsing mismatch, budget page, exports, account/sync.
Shared/API/sync	Accounts-and-Sync.md; Program.cs; SyncDbContext search; BudgetSyncEndpoints search; PaycheckApiClient search; BudgetMerger/DTO search results	Identity endpoints, PostgreSQL, migrations, paycheck/budget sync, tombstones, merge model, known limitations.
Tests	[Fact] search across PaycheckCalc.Tests	Coverage areas include sync API, budget, recurring bills, savings goals, bonus, gross-up, IRS/FICA explanations, exports, snapshots, state calculators, annual projection, and self-employment.
Prior roadmap PDF	Uploaded Paycheck App Roadmap.pdf	Baseline future-plan document used to identify completed/stale roadmap items.

Appendix B - Recommended Next 10 Moves

56. Add TaxYear and TaxDataVersion throughout input, context, snapshots, exports, and sync.
57. Build the Accuracy & Sources screen and source metadata model.
58. Fix Blazor state picker and decimal parsing.
59. Add MAUI YTD Social Security and Medicare wage fields.
60. Add privacy policy, terms, delete account, export data, and delete local data.
61. Add API rate limiting, HTTPS production guidance, and safe logging policy.
62. Add registry/source metadata/golden accuracy tests.
63. Add paycheck profiles and edit/duplicate saved paycheck flows.
64. Prototype paycheck calendar plus bill-to-paycheck assignment.
65. Prototype W-4 optimizer after annual projection and tax-year metadata are reliable.